Zach Lowe, EECS 348

Gen AI Code Comparison – Assignment 4

In this assignment, I chose to compare the way that the free browser models available for both Copilot and Claude fulfilled the following prompt while using the puzzle files and assignment 4 rubric provided on canvas as guidelines for them to follow:

Generate C++ code using objects, not functions, to implement the following requirements: Given a partially completed Sudoku puzzle, find the (or at least a) solution; if no solution exists, prove that. It is possible to solve a Sudoku puzzle entirely through logic. We're going to implement a brute-force search, in which we exhaustively try possibilities until we find a solution. Here's the idea: Suppose we have a Sudoku puzzle of which k of the 81 cells are unfilled. We identify an empty cell and deduce a list of candidates for what might go into that cell. If there are no such values (all digits are accounted for elsewhere in the same row, column, or grid), then we must have made a mistake in filling in the k cells and return a no-solution-found result. Otherwise, we try putting the first candidate value into the empty cell, and see if we can find a solution with the k+1 cells filled (i.e. a recursive call). If we get a no-solution-found to the first candidate, then we try the next, and so on, until we either find a solution (k == 81) or determine that no solution exists (all candidates report no solution, which again implies we took a wrong path somewhere earlier). You are given five files (puzzle1.txt through puzzle5.txt), each containing a Sudoku puzzle. Digits already filled in have been placed, and an underscore character (_) is used to indicate a blank square. Write a recursive object-oriented program to find the solution to the five puzzle files. The output for each solution, should include: The puzzle file name (e.g., puzzle1.txt). The puzzle from the file printed out. The solution to the puzzle printed out or if no solution is found, "No solution found" printed out. The solution may not be unique; if there is more than one solution to the puzzle, your program should print them all out. To help you debug your program, solution1.txt is a solution to puzzle1.txt.

```
/*
 * Program Name : sudoku.cpp
 * Description  : Object-oriented, recursive brute-force Sudoku solver. For each
 *                puzzle file it prints the file name, the original puzzle, and
 *                EVERY solution found (or "No solution found").
 * Inputs       : puzzle1.txt ... puzzle5.txt (9 rows of 9 tokens; digits 1-9,
 *                '_' for a blank cell), or file names given on the command line.
 * Output       : Console output: file name, puzzle, solution(s), solution count.
 * Collaborators: None (algorithm taken from the assignment description).
 *                Code generated with help from Claude (Anthropic GenAI).
 * Author       : <YOUR FULL NAME>
 * Created      : 10/14/2026   Last revised: 10/14/2026
 */
```

```cpp
#include <iostream>   // for std::cout / std::cerr output
#include <fstream>    // for reading puzzle files
#include <sstream>    // for splitting lines into tokens
#include <string>     // for std::string
#include <vector>     // for std::vector (list of candidates)

// Class that represents one Sudoku board and knows how to solve itself.
class SudokuPuzzle {
private:
    static const int N = 9;        // board is N x N
    static const int BOX = 3;      // each sub-grid is BOX x BOX
    int grid[N][N];                // 0 means empty, 1-9 are filled values
    long solutionCount;            // number of solutions found so far

    // Collects the digits that can legally go in cell (row, col).
    std::vector<int> candidates(int row, int col) const {
        bool used[N + 1] = {false};                    // used[d] true if digit d
is blocked
        for (int i = 0; i < N; ++i) {                  // scan the row and the
column
            used[grid[row][i]] = true;                 // digit already in this
row
            used[grid[i][col]] = true;                 // digit already in this
column
        }
        int br = (row / BOX) * BOX;                    // top row of this 3x3 box
        int bc = (col / BOX) * BOX;                    // left column of this 3x3
box
        for (int r = 0; r < BOX; ++r)                  // scan the 3x3 box
            for (int c = 0; c < BOX; ++c)
                used[grid[br + r][bc + c]] = true;     // digit already in this
box
        std::vector<int> result;                       // list of allowed digits
        for (int d = 1; d <= N; ++d)                   // try every digit 1-9
            if (!used[d]) result.push_back(d);         // keep the ones not
blocked
        return result;                                 // may be empty -> dead
end
    }

    // Finds the first empty cell; returns false if the board is full.
    bool findEmpty(int &row, int &col) const {
        for (int r = 0; r < N; ++r)                    // go through rows
```

```
            for (int c = 0; c < N; ++c)                 // go through columns
                if (grid[r][c] == 0) {                  // empty cell found
                    row = r;                            // report its row
                    col = c;                            // report its column
                    return true;                        // an empty cell exists
                }
        return false;                                   // no empty cells: k == 81
    }

    // Checks that the given clues do not already break a Sudoku rule.
    bool givensAreValid() const {
        for (int r = 0; r < N; ++r)                     // check every cell
            for (int c = 0; c < N; ++c) {
                int v = grid[r][c];                     // value in this cell
                if (v == 0) continue;                   // skip blanks
                for (int i = 0; i < N; ++i) {           // compare with row/column
mates
                    if (i != c && grid[r][i] == v) return false;  // duplicate in
row
                    if (i != r && grid[i][c] == v) return false;  // duplicate in
column
                }
                int br = (r / BOX) * BOX;               // box origin row
                int bc = (c / BOX) * BOX;               // box origin column
                for (int rr = br; rr < br + BOX; ++rr)          // compare within
box
                    for (int cc = bc; cc < bc + BOX; ++cc)
                        if ((rr != r || cc != c) && grid[rr][cc] == v)
                            return false;               // duplicate in box
            }
        return true;                                    // no conflicts found
    }

    // Recursive brute-force search. Each call fills one more empty cell.
    void solve() {
        int row = 0, col = 0;                           // location of the next
empty cell
        if (!findEmpty(row, col)) {                     // base case: all 81 cells
filled
            ++solutionCount;                            // count this solution
            std::cout << "Solution " << solutionCount << ":\n";  // label it
            print();                                    // print the solved board
            return;                                     // go back to look for
more
```

```
        }
        std::vector<int> options = candidates(row, col);  // legal digits for
this cell
        for (size_t i = 0; i < options.size(); ++i) {  // try each candidate in
order
            grid[row][col] = options[i];               // place the candidate
            solve();                                   // recurse with k+1 cells
filled
            grid[row][col] = 0;                        // backtrack: undo the
placement
        }
        // If options was empty, or all failed, we simply return (no solution on
this path).
    }

public:
    // Constructor: start with an empty board and zero solutions.
    SudokuPuzzle() : solutionCount(0) {
        for (int r = 0; r < N; ++r)                    // clear every row
            for (int c = 0; c < N; ++c)
                grid[r][c] = 0;                        // 0 = empty
    }

    // Reads the board from a file. Returns false if the file can't be read.
    bool load(const std::string &filename) {
        std::ifstream in(filename.c_str());            // open the file
        if (!in) return false;                         // report failure to open
        for (int r = 0; r < N; ++r)                    // read 9 rows
            for (int c = 0; c < N; ++c) {              // of 9 tokens each
                std::string tok;                       // one whitespace-
separated token
                if (!(in >> tok)) return false;        // file too short
                grid[r][c] = (tok[0] == '_') ? 0 : (tok[0] - '0');  // '_' -> 0,
else digit
            }
        return true;                                   // board loaded
    }

    // Prints the board using the same format as the input files.
    void print() const {
        for (int r = 0; r < N; ++r) {                  // each row
            for (int c = 0; c < N; ++c) {              // each cell
                if (grid[r][c] == 0) std::cout << "_ ";            // blank cell
```

```
                else std::cout << grid[r][c] << " ";              // filled cell
            }
            std::cout << "\n";                          // end of row
        }
        std::cout << "\n";                              // blank line after the
board
    }

    // Public entry point: runs the search and reports the results.
    void solveAll() {
        solutionCount = 0;                              // reset the counter
        if (givensAreValid()) solve();                  // only search if clues
are consistent
        if (solutionCount == 0)                         // nothing found
            std::cout << "No solution found\n\n";       // report failure
        else
            std::cout << "Solutions found: " << solutionCount << "\n\n";  //
report count
    }
};

int main(int argc, char *argv[]) {
    std::vector<std::string> files;                     // list of puzzle files to
process
    if (argc > 1) {                                     // use command-line names
if given
        for (int i = 1; i < argc; ++i) files.push_back(argv[i]);
    } else {                                            // otherwise default to
the five files
        for (int i = 1; i <= 5; ++i) {
            std::ostringstream name;                    // build "puzzleN.txt"
            name << "puzzle" << i << ".txt";
            files.push_back(name.str());
        }
    }
    for (size_t i = 0; i < files.size(); ++i) {         // process each file
        std::cout << "==== " << files[i] << " ====\n";  // print the file name
        SudokuPuzzle puzzle;                            // create a puzzle object
        if (!puzzle.load(files[i])) {                   // try to read the file
            std::cout << "Could not read " << files[i] << "\n\n";  // report
problem
            continue;                                   // move on to the next
file
        }
```

```
        std::cout << "Puzzle:\n";                    // label the original
puzzle
        puzzle.print();                              // show the puzzle
        puzzle.solveAll();                           // solve and print all
solutions
    }
    return 0;                                        // success
}
```

Figure 1 (Claude Initial Code)

```
/*****************************************************************************
* Program: Sudoku Solver
* Description:
*   Reads Sudoku puzzles from puzzle1.txt through puzzle5.txt and uses
*   recursive backtracking to find and print every solution.
*
* Inputs:
*   puzzle1.txt
*   puzzle2.txt
*   puzzle3.txt
*   puzzle4.txt
*   puzzle5.txt
*
* Output:
*   Original puzzle and all solutions found, or "No solution found".
*
* Author: Zach Lowe
* Collaborators:
*   Microsoft Copilot (code generation)
*
* Creation Date: 10/14/2026
*****************************************************************************/

#include <iostream>
#include <fstream>
#include <vector>
#include <string>

using namespace std;

/*****************************************************************************
```

```
* Class: SudokuBoard
* Purpose:
*   Stores and manages a Sudoku board.
*********************************************************************/
class SudokuBoard
{
private:
    int board[9][9];

public:

    SudokuBoard()
    {
        for (int row = 0; row < 9; row++)
        {
            for (int col = 0; col < 9; col++)
            {
                board[row][col] = 0;
            }
        }
    }

    bool loadPuzzle(const string& filename)
    {
        ifstream inputFile(filename);

        if (!inputFile)
        {
            return false;
        }

        char value;

        for (int row = 0; row < 9; row++)
        {
            for (int col = 0; col < 9; col++)
            {
                inputFile >> value;

                if (value == '_')
                {
                    board[row][col] = 0;
                }
```

```
            else
            {
                board[row][col] = value - '0';
            }
        }
    }

    inputFile.close();
    return true;
}

void printBoard() const
{
    for (int row = 0; row < 9; row++)
    {
        for (int col = 0; col < 9; col++)
        {
            if (board[row][col] == 0)
            {
                cout << "_ ";
            }
            else
            {
                cout << board[row][col] << " ";
            }
        }

        cout << endl;
    }
}

int getValue(int row, int col) const
{
    return board[row][col];
}

void setValue(int row, int col, int value)
{
    board[row][col] = value;
}

bool isEmpty(int row, int col) const
{
```

```
        return board[row][col] == 0;
    }

    bool findEmptyCell(int& row, int& col) const
    {
        for (row = 0; row < 9; row++)
        {
            for (col = 0; col < 9; col++)
            {
                if (board[row][col] == 0)
                {
                    return true;
                }
            }
        }

        return false;
    }

    bool isValidPlacement(int row, int col, int number) const
    {
        // Check row
        for (int currentCol = 0; currentCol < 9; currentCol++)
        {
            if (board[row][currentCol] == number)
            {
                return false;
            }
        }

        // Check column
        for (int currentRow = 0; currentRow < 9; currentRow++)
        {
            if (board[currentRow][col] == number)
            {
                return false;
            }
        }

        // Check 3x3 subgrid
        int startRow = (row / 3) * 3;
        int startCol = (col / 3) * 3;
```

```
        for (int currentRow = startRow; currentRow < startRow + 3; currentRow++)
        {
            for (int currentCol = startCol; currentCol < startCol + 3;
currentCol++)
            {
                if (board[currentRow][currentCol] == number)
                {
                    return false;
                }
            }
        }

        return true;
    }
};

/*****************************************************************************
* Class: SudokuSolver
* Purpose:
*   Uses recursive backtracking to solve Sudoku puzzles.
*****************************************************************************/
class SudokuSolver
{
private:
    int solutionCount;

public:

    SudokuSolver()
    {
        solutionCount = 0;
    }

    void solve(SudokuBoard& board)
    {
        int row;
        int col;

        // Base case: no empty cells remain
        if (!board.findEmptyCell(row, col))
        {
            solutionCount++;
```

```
            cout << "\nSolution #" << solutionCount << endl;
            board.printBoard();
            cout << endl;

            return;
        }

        // Try digits 1 through 9
        for (int value = 1; value <= 9; value++)
        {
            if (board.isValidPlacement(row, col, value))
            {
                board.setValue(row, col, value);

                // Recursive call
                solve(board);

                // Backtrack
                board.setValue(row, col, 0);
            }
        }
    }

    int getSolutionCount() const
    {
        return solutionCount;
    }
};

/*****************************************************************************
* Main Program
*****************************************************************************/
int main()
{
    vector<string> files =
    {
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
    };
```

```
    for (string filename : files)
    {
        SudokuBoard board;

        cout << "======================================" << endl;
        cout << "Puzzle File: " << filename << endl;
        cout << "======================================" << endl;

        if (!board.loadPuzzle(filename))
        {
            cout << "Unable to open file.\n\n";
            continue;
        }

        cout << "\nOriginal Puzzle:\n";
        board.printBoard();

        SudokuSolver solver;

        cout << "\nSolutions:\n";
        solver.solve(board);

        if (solver.getSolutionCount() == 0)
        {
            cout << "No solution found" << endl;
        }

        cout << endl;
    }

    return 0;
}
```

Figure 2 (Copilot Initial Code)

Firstly, in terms of correctness, Claude performs better than Copilot due to its additional validation of the starting puzzle and its checks to ensure that the input file contains enough values to fill the board. Secondly, in terms of execution time, Claude has a slight advantage because it generates a list of valid candidates before trying them, whereas Copilot checks each digit individually. Thirdly, in terms of space complexity, Copilot is slightly more efficient because it does not need to create candidate vectors

during the recursive search, although both implementations have the same overall space complexity. Lastly, in terms of maintainability, Claude performs better due to its clearly separated helper methods, extensive comments, and support for command-line file inputs. Given the comparisons/analysis above, I have chosen to continue with Claude's code for the remainder of this assignment; primarily because of the improved input validation, candidate generation, and maintainability.

With the revisions, my focus was on improving correctness and maintainability. The existing execution time and space complexity were already acceptable for the purpose of this program, so I prioritized improvements that would make the solver more reliable and easier to maintain. To improve correctness, I updated the program to validate every cell in the puzzle file, rejecting malformed input with an error message identifying the offending row and column. I also added a final check confirming that each completed board is a valid solution before it is printed. To improve maintainability, I replaced the separate starting-puzzle validation routine with a single rule-checking method that can validate both the initial clues and the completed board.